

Модулі. Стандартна бібліотека. Імпорт

Модулі та атрибути

Реальні програми зазвичай не записані в одному файлі. Вони складаються з багатьох пов'язаних між собою модулів — файлів з розширенням `.py`, іменування яких підлягає правилам іменування змінних. З іншого боку, модуль є простором імен¹. Імена всередині модуля називаються атрибутами². Зазвичай атрибутами модулів є змінні, функції та класи.

Атрибут завжди пов'язаний з певним об'єктом (у нашому випадку модулем). Цей зв'язок записується у точковій нотації, синтаксис якої:

```
<ім'я_об'єкта(модуля)>.<ім'я_атрибута>
```

Наприклад, якщо файл `todo.py` містить змінні з запланованими завданнями:

```
task_1 = 'Прочитати книжку'  
task_2 = 'Посадити дерево'
```

то наступний фрагмент коду, який записаний в іншому файлі, виведе текст другого завдання (значення змінної `task_2`):

```
print(todo.task_2)    # Посадити дерево
```

Насправді, якщо запустити цей фрагмент, то з'явиться повідомлення про помилку:

```
NameError: name 'todo' is not defined
```

Справа в тому, що інтерпретатору під час виконання інструкції нічого не відомо про модуль `todo` (файл `todo.py`), а отже, і про його атрибути. Модуль перед використанням треба імпортувати. Для цього призначена інструкція `import`, яку зазвичай пишуть на початку коду:

```
import todo  
print(todo.task_2)
```

¹ Простір імен містить усі імена об'єктів, яким присвоюється значення у модулі, крім імен всередині інструкцій `def` і `class`.

² Атрибути мають не тільки модулі, а будь-які інші об'єкти

Якщо атрибутом модуля є функція, то її виклик здійснюється теж за правилом точкової нотації:

```
<ім'я_модуля>.<ім'я_функції>(<аргумент1>, <аргумент2>, ...)
```

Для того, щоб інструкція `import` змогла знайти модуль (у нашому випадку це файл `todo.py`), він повинен бути збережений у тій же папці, що й файл основної програми, яка імпортує цей модуль. Якщо модуля там немає, інтерпретатор намагатиметься знайти його у папках стандартної бібліотеки та папках сховища сторонніх пакетів. У разі відсутності модуля і там, інтерпретатор видасть помилку імпорту³:

```
ImportError: No module named 'todo'
```

Різновиди інструкції `import`

Описаний вище спосіб імпорту модуля, синтаксис якого

```
import <ім'я_модуля>
```

не єдиний. Є інші способи імпорту, яким відповідають інші варіанти синтаксису доступу до атрибутів.

Можна уникнути точкової нотації, використавши синтаксис

```
from <ім'я_модуля> import *  
<ім'я_атрибута> # доступ без точкової нотації
```

Цей спосіб імпортує усі доступні в модулі атрибути. Якщо використовуються лише деякі, можна їх імпортувати окремо:

```
from <ім'я_модуля> import <атрибут1>, <атрибут2>  
<атрибут1> # доступ без точкової нотації  
<атрибут2>() # доступ без точкової нотації
```

Модулю або його атрибуту можна присвоювати нові імена, використовуючи `as`:

```
import <ім'я_модуля> as <нове_ім'я_модуля>  
<нове_ім'я_модуля>.<ім'я_атрибута> # доступ з використанням точкової нотації  
  
from <ім'я_модуля> import <ім'я_атрибута> as <нове_ім'я_атрибута>  
<нове_ім'я_атрибута> # доступ без точкової нотації
```

³ В інструкції `import` можна використовувати відносні шляхи, які вказують місце розташування модуля

Вибір способу імпорту залежить від конкретної ситуації. Наприклад, якщо різні модулі містять однойменні атрибути, для уникнення неоднозначності у коді

```
from mod_1 import f      # імпорт функції f() з модуля mod_1
from mod_2 import f      # імпорт функції з таким самим іменем f() з модуля mod_2
f()                      # буде викликана функція модуля mod_2
```

доцільніше використовувати один з наступних варіантів:

```
# спосіб 1
from mod_1 import f as f1
from mod_2 import f as f2
f1()                      # виклик функції з першого модуля
f2()                      # виклик функції з другого модуля

# спосіб 2
import mod_1, mod_2
mod_1.f()                 # виклик функції з першого модуля
mod_2.f()                 # виклик функції з другого модуля
```

Стандартна бібліотека

Стандартна бібліотека Python містить десятки модулів з готовим якісним кодом для роботи з різноманітними об'єктами та розширює функціонал мови. Нижче наведено приклади використання декількох з них. Для закріплення знань з імпорту модулів у наведених прикладах використовуються різні способи імпорту.

Отримати перелік доступних атрибутів модуля можна, ввівши в інтерактивному сеансі послідовність команд:

```
>>> import <ім'я_модуля>
>>> dir(<ім'я_модуля>)      # список імен атрибутів
>>> help(<ім'я_модуля>)    # повна документація про модуль
```

або скористатися доступною у відкритих джерелах документацією.

Модуль math

Математичний модуль math містить математичні константи та функції для здійснення основних арифметичних, тригонометричних, логарифмічних та інших операцій, а саме:

```

acos(x)          # повертає арккосинус числа x (в радіанах)
acosh(x)         # гіперболічний арккосинус числа x
asin(x)          # арксинус числа x (в радіанах)
asinh(x)         # гіперболічний арксинус числа x
atan(x)          # арктангенс числа x (в радіанах)
atan2(y,x)       # арктангенс частки y/x (в радіанах) з урахуванням чверті, в
                  # якій знаходиться точка (x,y)
atanh(x)         # гіперболічний арктангенс числа x
ceil(x)          # заокруглює число x до найближчого більшого цілого
copysign(x,y)    # повертає число, модуль якого дорівнює модулю числа x, а знак
                  # співпадає зі знаком числа y
cos(x)           # повертає косинус числа x (в радіанах)
cosh(x)          # гіперболічний косинус числа x
degrees(x)       # перетворює кут x з радіанної міри в градусну
e                # 2.718281828459045
erf(x)           # функція помилок
erfc(x)          # 1-erf(x)
exp(x)           # ex
expm1(x)         # ex - 1 (з кращою точністю, ніж exp(x)-1)
fabs(x)          # модуль (абсолютне значення) числа x
factorial(x)     # x! (факторіал числа x)
floor(x)         # заокруглює число x до найближчого меншого цілого
fmod(x,y)        # остача від ділення x на y
frexp(x)         # повертає дійсну мантису і цілу експоненту числа x у вигляді
                  # пари (m,e), такої що x = m * 2. ** e
fsum(seq)        # повертає суму елементів послідовності seq як дійсне число
gamma(x)         # значення гамма-функції в точці x
gcd(x,y)         # найбільший спільний дільник чисел x і y
hypot(x,y)       # гіпотенуза прямокутного трикутника з катетами x і y
inf              # нескінченність
isclose(x,y)     # перевіряє на близькість (регулюється іменованими аргументами
                  # rel_tol і abs_tol) числа x і y (повертає True або False4)
isfinite(x)      # перевіряє, чи число x є скінченним
isinf(x)         # перевіряє, чи число x є нескінченним
isnan(x)         # повертає True, якщо x не є числом, і False, якщо x - число
ldexp(x,i)       # повертає x * (2**i)
lgamma(x)        # натуральний логарифм абсолютного значення гамма-функції в
                  # точці x
log(x)           # натуральний логарифм числа x
log(x,y)         # логарифм за основою y числа x
log1p(x)         # натуральний логарифм числа 1+x
log2(x)          # логарифм за основою 2 числа x
modf(x)          # повертає дробову і цілу частину числа x
nan              # не число
pi              # 3.141592653589793
tau              # 6.283185307179586
pow(x,y)         # повертає x**y
radians(x)       # перетворює кут x з градусної міри в радіанну

```

```
remainder(x,y)    # повертає x-n*y, де n – найближче до x ціле число
sin(x)            # синус числа x (в радіанах)
sinh(x)           # гіперболічний синус числа x
sqrt(x)           # квадратний корінь числа x
tan(x)            # тангенс числа x (в радіанах)
tanh(x)           # гіперболічний тангенс числа x
trunk(x)          # відтинає дробову частину числа x (повертає ціле число)
```

Розглянемо використання окремих функцій модуля math на прикладі програми для визначення кута між векторами:

```
import math

x1, y1 = float(input('x1 = ')), float(input('y1 = '))
x2, y2 = float(input('x2 = ')), float(input('y2 = '))

m1 = math.hypot(x1, y1)          # модуль (довжина) першого вектора
m2 = math.hypot(x2, y2)          # модуль (довжина) другого вектора
sp = x1*x2 + y1*y2               # скалярний добуток векторів
rad = math.acos(sp/m1/m2)         # кут у радіанах
deg = math.degrees(rad)           # кут у градусах

print('Кут між векторами у радіанах:', rad)
print('Кут між векторами у градусах:', deg)
```

Модуль random

Модуль random містить набір функцій для генерування випадкових елементів, зокрема:

```
randint(x, y)        # повертає випадкове ціле число з проміжку [x; y]
random()              # повертає випадкове дійсне число з проміжку [0; 1]
shuffle(L)            # перемішує елементи списку L
uniform(x, y)         # повертає випадкове дійсне число з проміжку [x, y]
```

Нижче наведено код програми, яка генерує випадкове ціле число від 1 до 100 і пропонує користувачу відгадати його. Результатом виконання програми є число, яке вказує, на скільки одиниць користувач помилився (0 означає, що користувач вгадав число):

```
from random import *

generated_number = randint(1, 100)
print('Відгадайте згенероване число (від 1 до 100)')
```

```
your_number = int(input('Введіть Ваш варіант: '))
print('Близькість Вашої здогадки:', abs(generated_number - your_number))
```

Модуль fractions

Модуль fractions пропонує тип Fraction для роботи з раціональними числами. Продemonструємо деякі можливості цього модуля на такому прикладі:

```
from fractions import Fraction as F

x = F(2, 7)          # записуємо у змінну x дріб (об'єкт Fraction) 2/7
y = F(14, 4)         # записуємо у змінну y дріб 14/4
print(x, y)          # 2/7 7/2 (другий дріб автоматично скорочується)
print(x-y, x*y)      # -45/14 1 (дії над дробами)
print(F('0.75'))    # 3/4 (створення дробу з рядка)
```

Модуль time

Розглянемо однойменну функцію time() модуля time, яка повертає кількість секунд, яка пройшла від 0 годин 0 хвилин 1 січня 1970 року (в середовищі програмістів цей момент часу називається "початок Епохи") до системного часу комп'ютера в момент її виклику. Часто її використовують для отримання часу виконання програми. Визначимо, для прикладу, скільки часу потрібно на генерування одного мільярда випадкових дійсних чисел від 0 до 1:

```
from time import time
from random import random as rnd

start = time()          # фіксація початку виконання операцій
for i in range(1000000000):
    rnd()               # повторення дії мільярд разів7
                        # дія, яка повторюється8
finish = time()         # фіксація завершення виконання операцій
print(round(finish - start, 1))  # виведення часу виконання
```

Інші модулі

- collections — пропонує структури даних, на додачу до вбудованих списків, кортежів, словників і множин;
- сору — надає розширені можливості для створення копій структур;

- `datetime` — містить функції для роботи з датами і часом;
 - `itertools` — набір інструментів для реалізації ітерацій;
 - `multiprocessing` — дає змогу запускати додаткові процеси та розподіляти обчислення між процесорами;
 - `pickle` — дає змогу зберігати у файл та зчитувати з файлу об'єкти Python (серіалізація);
 - `re` — забезпечує роботу з регулярними виразами (regular expressions);
 - `statistics` — містить функції для статистичного аналізу даних;
 - `tkinter` — надає інструментарій для створення графічних інтерфейсів;
 - `turtle` — надає графічний інструментарій для створення простих двовимірних рисунків;
 - `os` — містить функції для роботи з файловою системою: зміна поточної директорії, отримання списку файлів і папок за вказаним шляхом, створення та вилучення папок, перейменування папок і файлів, перевірка існування файлу чи папки, отримання розміру файла та ін.
 - `pprint` — дає змогу відображати на екрані структури даних у зручному для сприйняття вигляді;
 - `sys` — інструментарій для системного програмування;
 - `threading` — пропонує високорівневий інтерфейс для реалізації багатопотоковості в операційній системі;
 - `unittest` — пропонує інструменти для автоматизованого тестування коду;
 - `urllib` — модуль для роботи з HTTP.
-